

# Perancangan dan Implementasi Basis Data Terdistribusi pada Platform E-Commerce “TokoKita” Menggunakan MongoDB Sharded Cluster dan Replica Set

Jefry Sunupurwa Asri, S.Kom., M.Kom.

.....

**WAHYU**

**20220801523**

**TEKNIK INFORMATIKA**



# LATAR BELAKANG MASALAH

## 1 Kapasitas

Volume data pesanan tumbuh melebihi kapasitas penyimpanan dan memori satu server basis data tunggal.

## 2 Ketersediaan

Kegagalan satu server berarti seluruh layanan toko lumpuh — single point of failure tanpa mekanisme failover.

## 3 Kinerja

Lonjakan trafik jam sibuk (flash sale) meningkatkan latensi baca-tulis karena seluruh beban terpusat.

## 4 Skalabilitas

Scale-up (memperbesar satu mesin) mahal dan terbatas; dibutuhkan scale-out menambah mesin secara horizontal.





# RUMUSAN MASALAH DAN TUJUAN SOLUSI

## Rumusan Masalah

- Bagaimana merancang arsitektur basis data terdistribusi yang menampung pertumbuhan data pesanan?
- Bagaimana memilih shard key agar data merata dan query tetap efisien?
- Bagaimana sistem tetap tersedia saat sebuah node gagal?
- Bagaimana konsistensi dijaga pada operasi lintas dokumen (checkout)?

## Tujuan

- Membangun cluster MongoDB terdistribusi: 2 shard × replica set 3 node + config server + router mongos (10 node, Docker Compose).
- Mendemonstrasikan distribusi data otomatis, query tertarget vs broadcast, agregasi terdistribusi, dan transaksi multi-dokumen.
- Menguji failover otomatis sebagai bukti empiris ketersediaan tinggi.



# PERBANDINGAN SISTEM SEJENIS

| Aspek                         | MongoDB Sharded (dipilih)                | MySQL Tunggal + Replikasi | Apache Cassandra | PostgreSQL + Citus    |
|-------------------------------|------------------------------------------|---------------------------|------------------|-----------------------|
| Model data                    | Dokumen (JSON) — cocok pesanan bersarang | Relasional                | Wide-column      | Relasional            |
| Skalabilitas tulis            | ✓ Horizontal via sharding otomatis       | ✗ Terbatas 1 primary      | ✓ Sangat baik    | ✓ Baik                |
| Failover otomatis             | ✓ Election replica set (±detik)          | △ Perlu tooling tambahan  | ✓ Masterless     | △ Bergantung setup    |
| Transaksi ACID multi-dokumen  | ✓ Didukung (sejak 4.2 lintas shard)      | ✓ Kuat                    | ✗ Terbatas       | ✓ Kuat                |
| Query per pelanggan tertarget | ✓ Shard key hashed(customer_id)          | —                         | ✓ Partition key  | ✓ Distribution column |
| Kemudahan simulasi lokal      | ✓ Image resmi Docker, 10 node ringan     | ✓ Mudah                   | △ Berat          | △ Sedang              |

Kombinasi dokumen fleksibel + sharding otomatis + replica set + transaksi ACID menjadikan MongoDB paling sesuai untuk beban kerja pesanan e-commerce studi kasus ini.





### Basis Data Terdistribusi

Kumpulan basis data yang terhubung logis, tersebar di banyak node, namun tampak sebagai satu sistem tunggal bagi pengguna (Özsu & Valduriez, 2020).

### Sharding (Fragmentasi Horizontal)

Setiap dokumen ditempatkan pada tepat satu shard berdasarkan shard key; MongoDB membagi data menjadi chunk yang diseimbangkan otomatis oleh balancer.

### Replica Set

Kelompok node dengan data identik: satu PRIMARY (tuliskan) dan beberapa SECONDARY. Kegagalan PRIMARY memicu election berbasis konsensus Raft dalam hitungan detik.

### Teorema CAP (Consistency, Availability, Partition Tolerance)

Saat terjadi partisi jaringan, sistem memilih konsistensi atau ketersediaan (Brewer, 2012). MongoDB berorientasi CP, dapat disetel per operasi via write/read concern.

# Mengapa Sharding + Replica Set, dan Bagaimana?

## WHY?

- Sharding memberi skalabilitas tulis linear: kapasitas bertambah dengan menambah shard, tanpa downtime.
- Replica set memberi ketersediaan tinggi: failover otomatis tanpa intervensi operator.
- Kombinasi keduanya memisahkan dua dimensi masalah — volume data (sharding) dan toleransi kegagalan (replikasi).

## HOW?

- Setiap shard adalah replica set 3 node; metadata chunk disimpan pada config server replica set.
- Router mongos menjadi satu-satunya titik kontak aplikasi: menargetkan query ke shard yang relevan lalu menggabungkan hasil.
- Shard key hashed(customer\_id) pada koleksi orders: distribusi merata + query riwayat pelanggan tertarget ke satu shard.

# METODOLOGI PENELITIAN

## 1. Analisis Kebutuhan

Identifikasi 5 kebutuhan (K-01-K-05): kapasitas, ketersediaan, akses per pelanggan, atomisitas checkout, isolasi beban analitik.

## 2. Perancangan

Topologi cluster 10 node, skema 3 koleksi, evaluasi 3 kandidat shard key untuk koleksi orders.

## 3. Implementasi

Provisi Docker Compose, inisialisasi replica set, konfigurasi sharding, seeding 5.000 pesanan sintetis, aplikasi demo pymongo.

## 4. Pengujian & Analisis

Lima skenario uji (U-1-U-5): distribusi chunk, targeted vs broadcast, failover, pemulihan node, atomisitas transaksi.

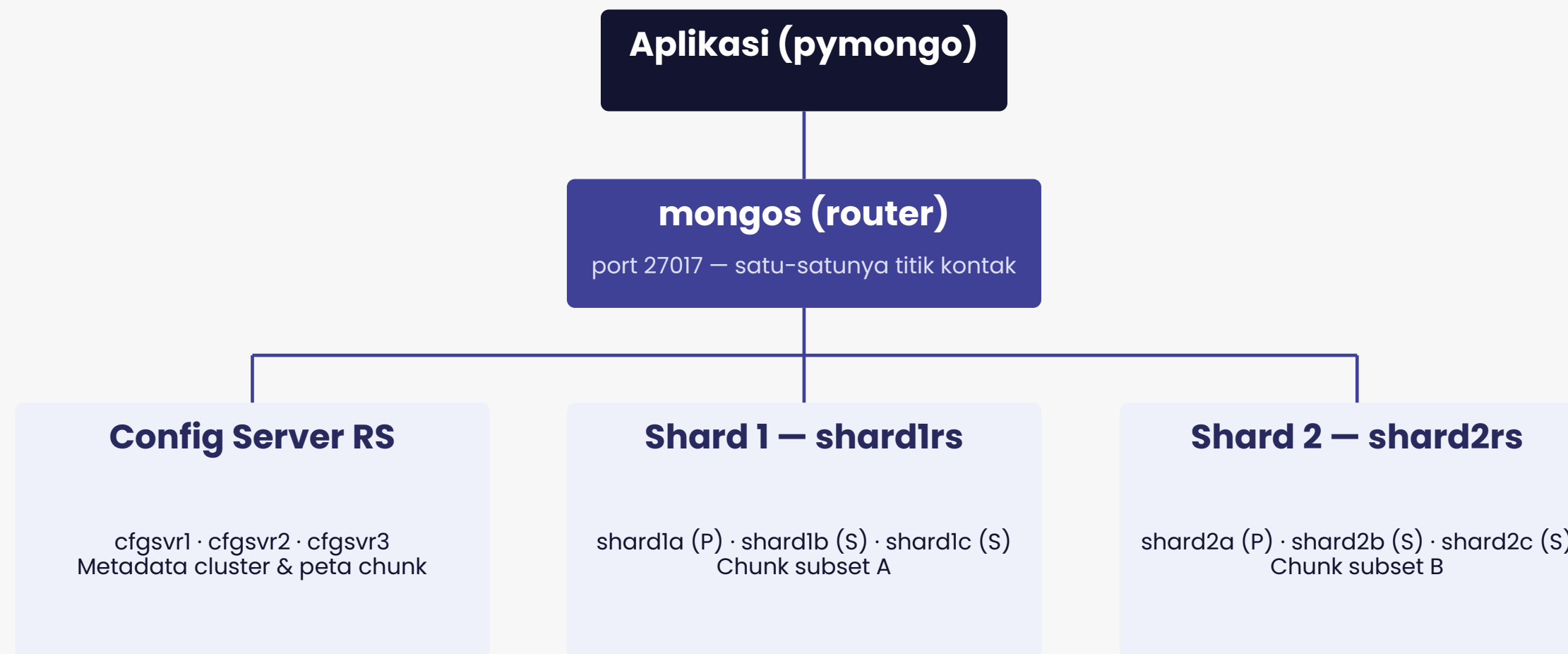
# Objek Studi: TokoKita & Kebutuhan Sistem

TokoKita adalah platform e-commerce dengan beban dominan: penulisan pesanan intensif saat jam sibuk, pembacaan riwayat pesanan per pelanggan, penelusuran katalog, dan laporan agregat penjualan.

| Kode | Kebutuhan                                              | Solusi Arsitektur                                                       |
|------|--------------------------------------------------------|-------------------------------------------------------------------------|
| K-01 | Data pesanan tumbuh melebihi kapasitas satu server     | Sharding koleksi orders ke 2 shard (dapat ditambah tanpa henti layanan) |
| K-02 | Layanan tetap hidup saat satu server mati              | Setiap shard = replica set 3 node dengan failover otomatis              |
| K-03 | Riwayat pesanan per pelanggan cepat diakses            | Shard key customer_id → query per pelanggan tertarget ke 1 shard        |
| K-04 | Checkout mengubah stok + membuat pesanan secara atomik | Transaksi multi-dokumen ACID lintas koleksi                             |
| K-05 | Laporan penjualan tidak mengganggu operasi utama       | Read preference secondaryPreferred untuk beban analitik                 |



# ARSITEKTUR SISTEM



*P = PRIMARY (menerima tulis) · S = SECONDARY (replika baca / kandidat failover)*

# Matriks Skenario Pengujian

| No  | Skenario                     | Cara Uji                                  | Hasil yang Diharapkan                                 |
|-----|------------------------------|-------------------------------------------|-------------------------------------------------------|
| U-1 | Distribusi data antar shard  | \$shardedDataDistribution setelah seeding | orders & products terbagi ≈50:50 antara kedua shard   |
| U-2 | Query tertarget vs broadcast | explain() dengan & tanpa shard key        | By customer_id → 1 shard; by status → semua shard     |
| U-3 | Failover otomatis            | docker stop node PRIMARY shard 1          | PRIMARY baru terpilih ±10 detik; aplikasi tetap jalan |
| U-4 | Pemulihan node               | docker start node yang dimatikan          | Kembali sebagai SECONDARY, sinkron via oplog          |
| U-5 | Atomisitas checkout          | Transaksi dengan qty melebihi stok        | Rollback otomatis; stok & pesanan tidak berubah       |

# Hasil Uji: Distribusi Data & Efisiensi Query

**On-Progress**

# Hasil Uji Failover & Analisis CAP

**On-Progress**

# Kesimpulan dan Saran

**On-Progress**





# TERIMA KASIH